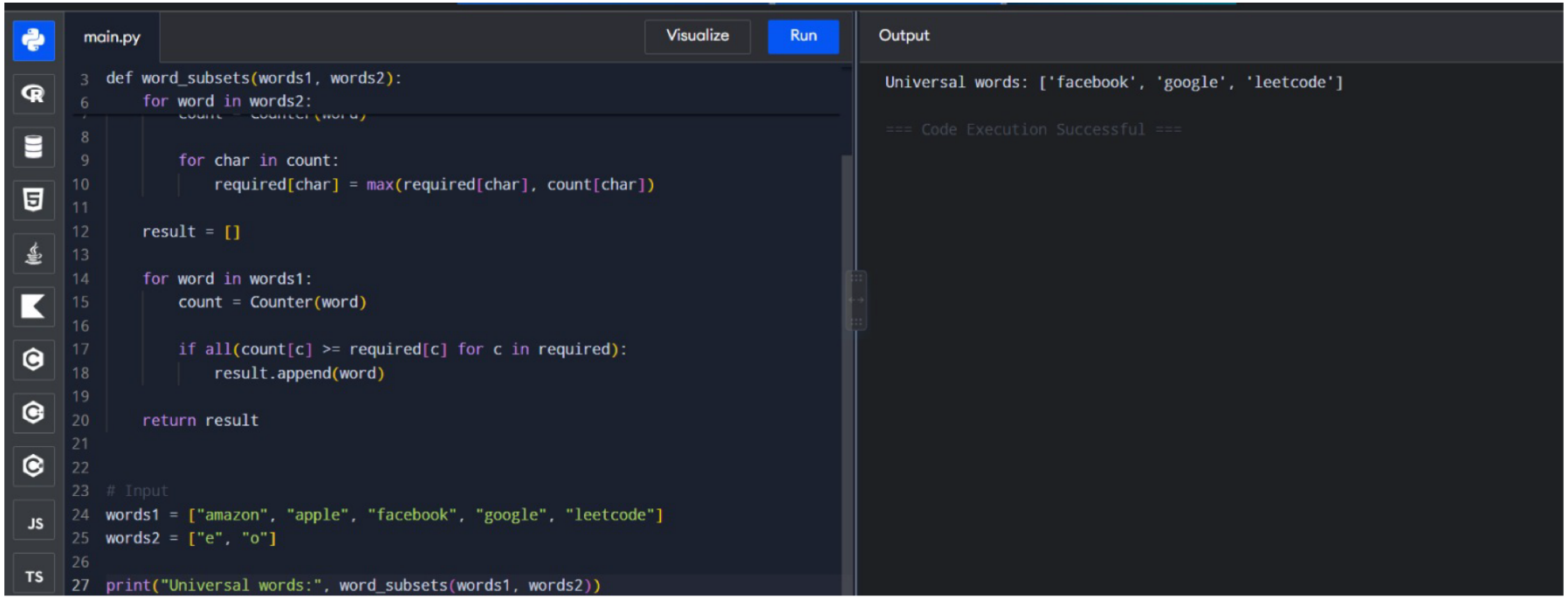
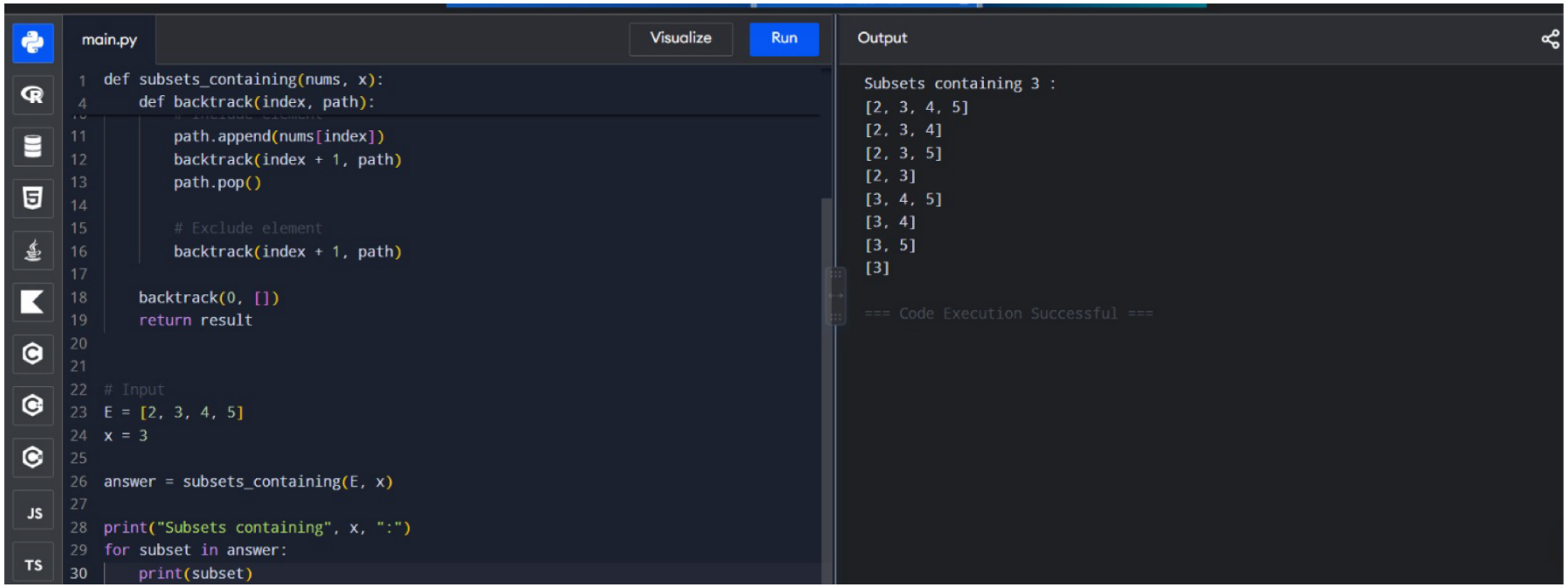
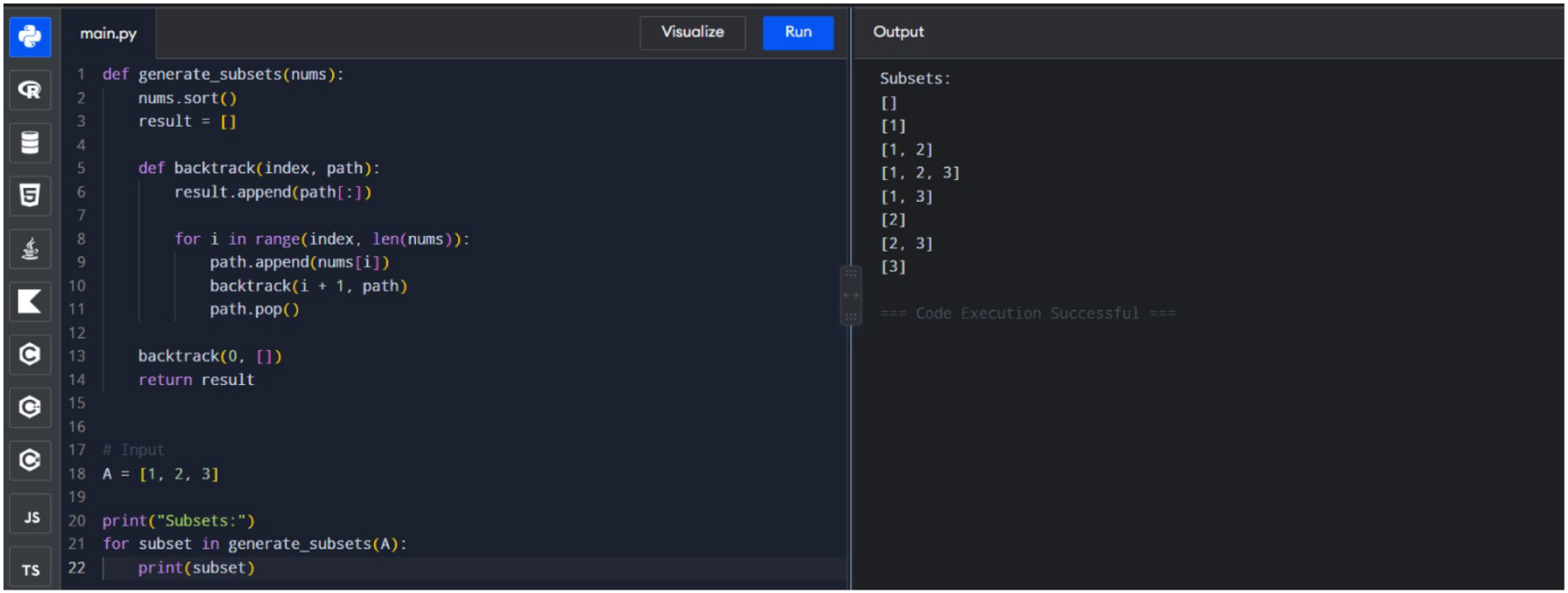
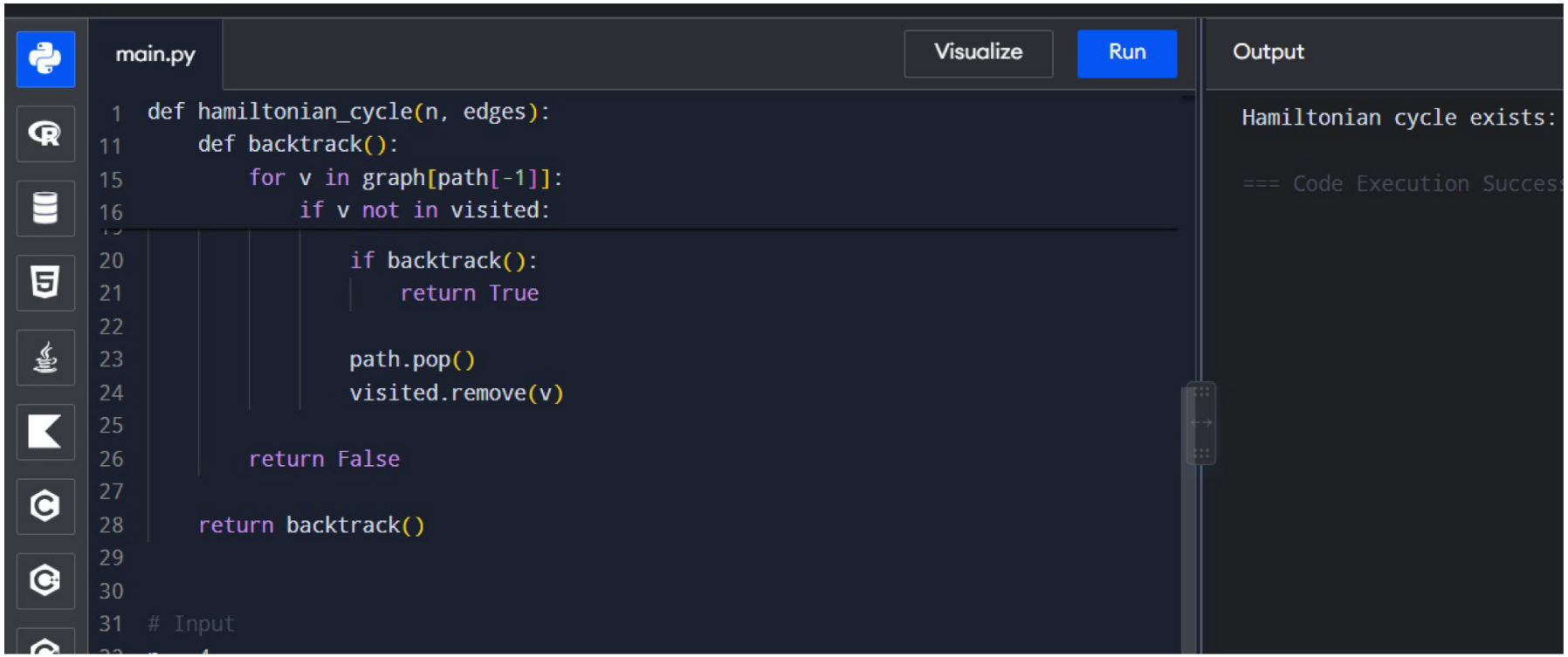
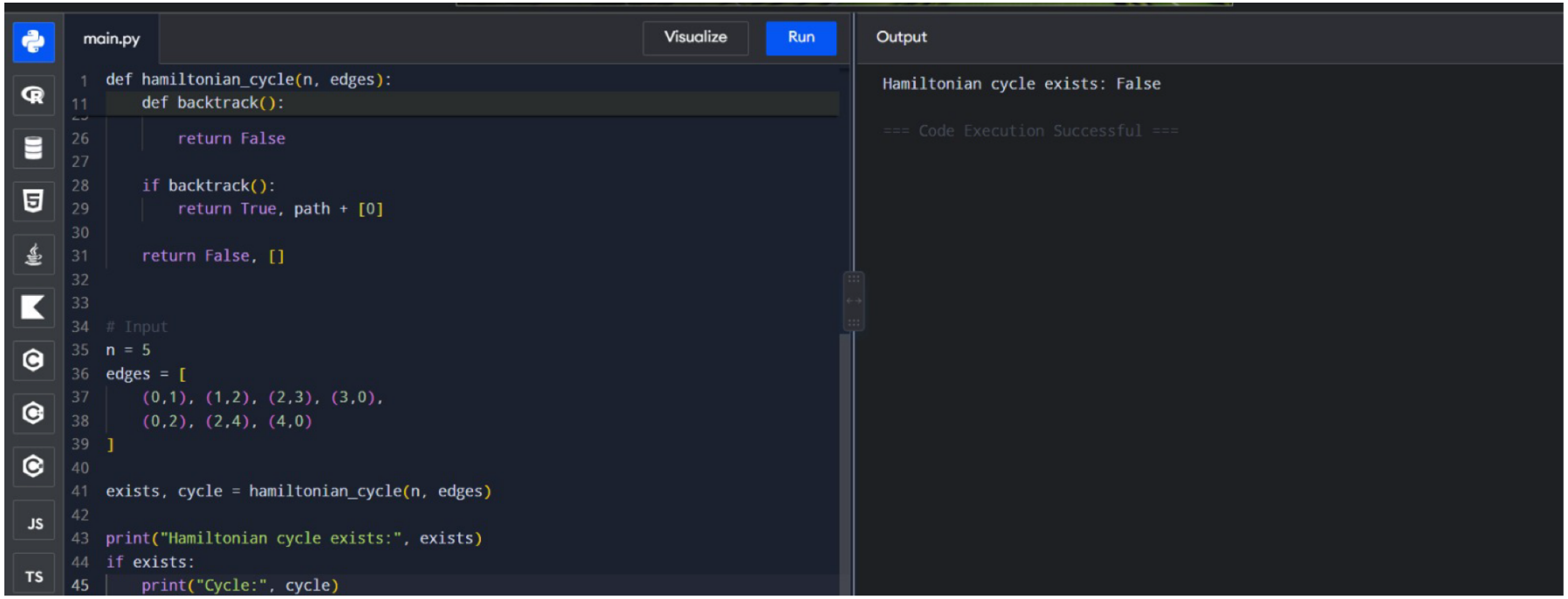
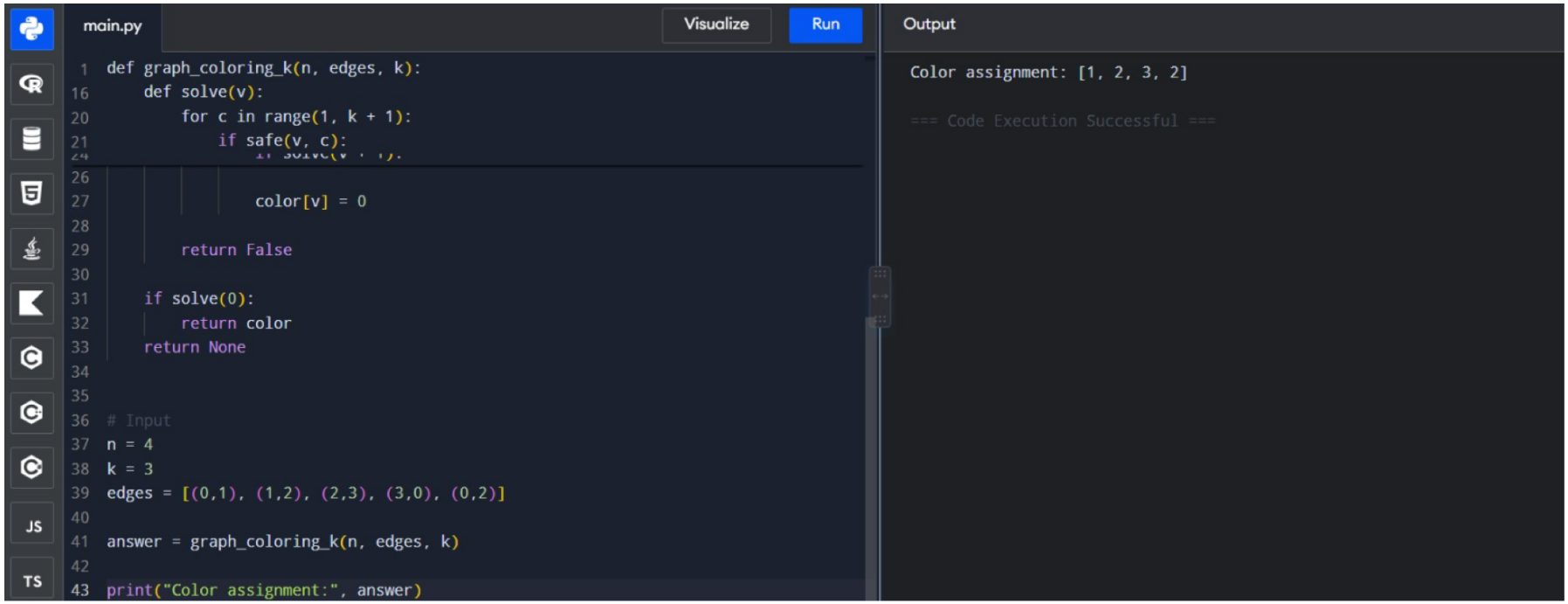

main.py
Visualize
Run
Output
def word_subsets(words1, words2):
for word in words2:
for char in count:
required[char] = max(required[char], count[char])
result = []
for word in words1:
count = Counter(word)
if all(count[c] >= required[c] for c in required):
result.append(word)
return result
# Input
words1 = ["amazon", "apple", "facebook", "google", "leetcode"]
words2 = ["e", "o"]
print("Universal words:", word_subsets(words1, words2))
Universal words: ['facebook', 'google', 'leetcode']
=== Code Execution Successful ===

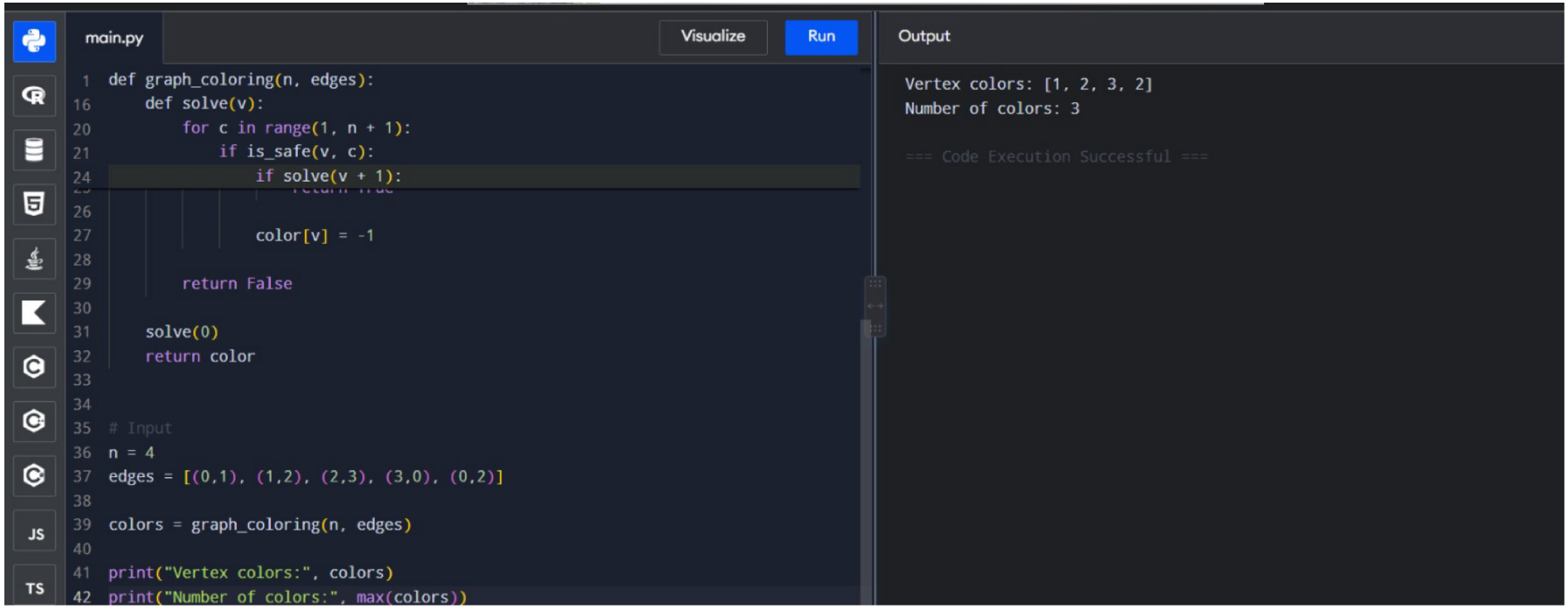

main.py
Visualize
Run
Output
def subsets_containing(nums, x):
def backtrack(index, path):
path.append(nums[index])
backtrack(index + 1, path)
path.pop()
# Exclude element
backtrack(index + 1, path)
backtrack(0, [])
return result
# Input
E = [2, 3, 4, 5]
x = 3
answer = subsets_containing(E, x)
print("Subsets containing", x, ":")
for subset in answer:
print(subset)
Subsets containing 3 :
[2, 3, 4, 5]
[2, 3, 4]
[2, 3, 5]
[2, 3]
[3, 4, 5]
[3, 4]
[3, 5]
[3]
=== Code Execution Successful ===

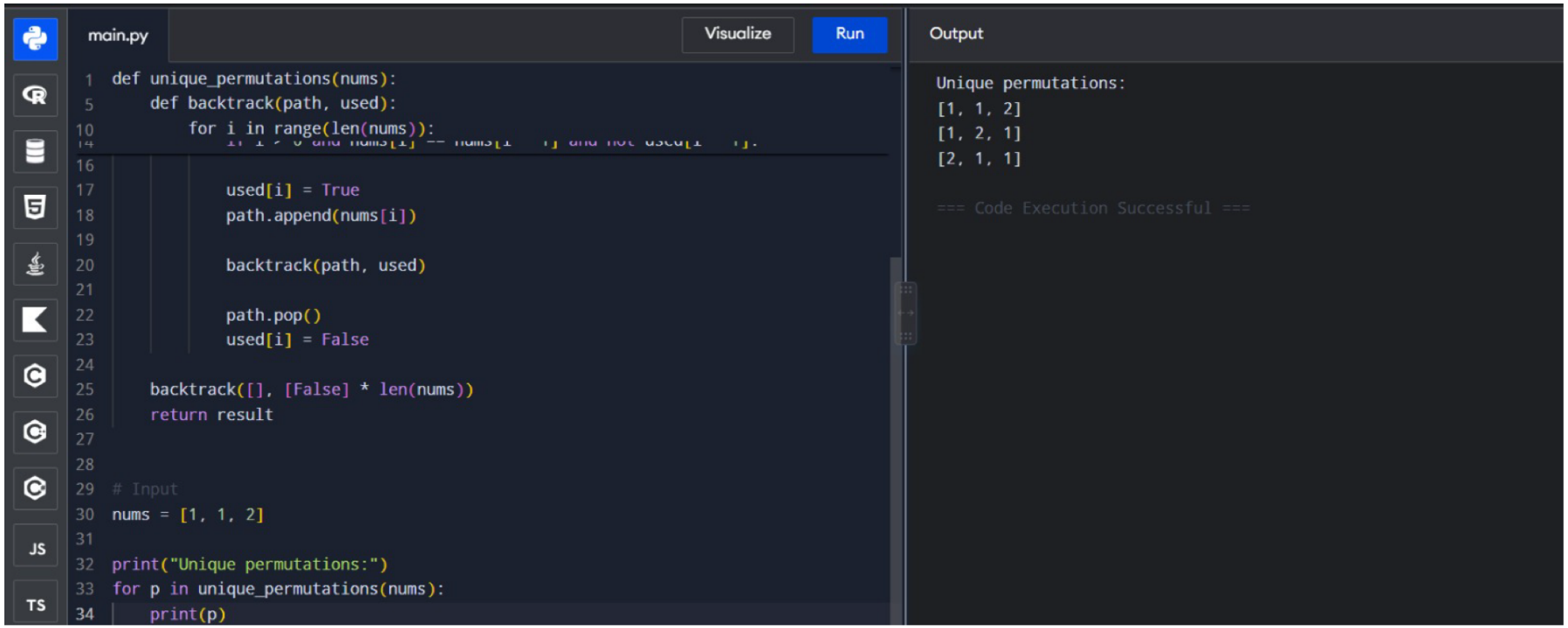

main.py
Visualize
Run
Output
def generate_subsets(nums):
nums.sort()
result = []
def backtrack(index, path):
result.append(path[:])
for i in range(index, len(nums)):
path.append(nums[i])
backtrack(i + 1, path)
path.pop()
backtrack(0, [])
return result
# Input
A = [1, 2, 3]
print("Subsets:")
for subset in generate_subsets(A):
print(subset)
Subsets:
[]
[1]
[1, 2]
[1, 2, 3]
[1, 3]
[2]
[2, 3]
[3]
=== Code Execution Successful ===

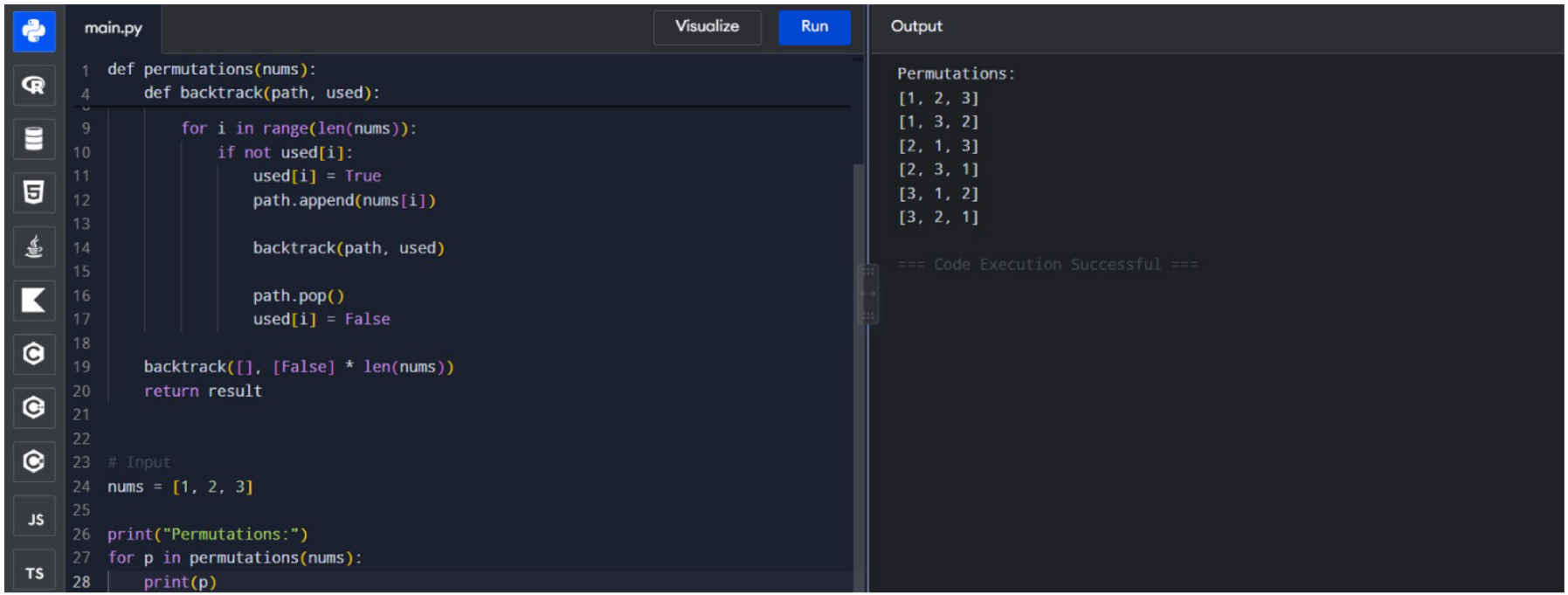

main.py
Visualize
Run
Output
def hamiltonian_cycle(n, edges):
def backtrack():
for v in graph[path[-1]]:
if v not in visited:
if backtrack():
return True
path.pop()
visited.remove(v)
return False
return backtrack()
# Input
Hamiltonian cycle exists:
=== Code Execution Succes

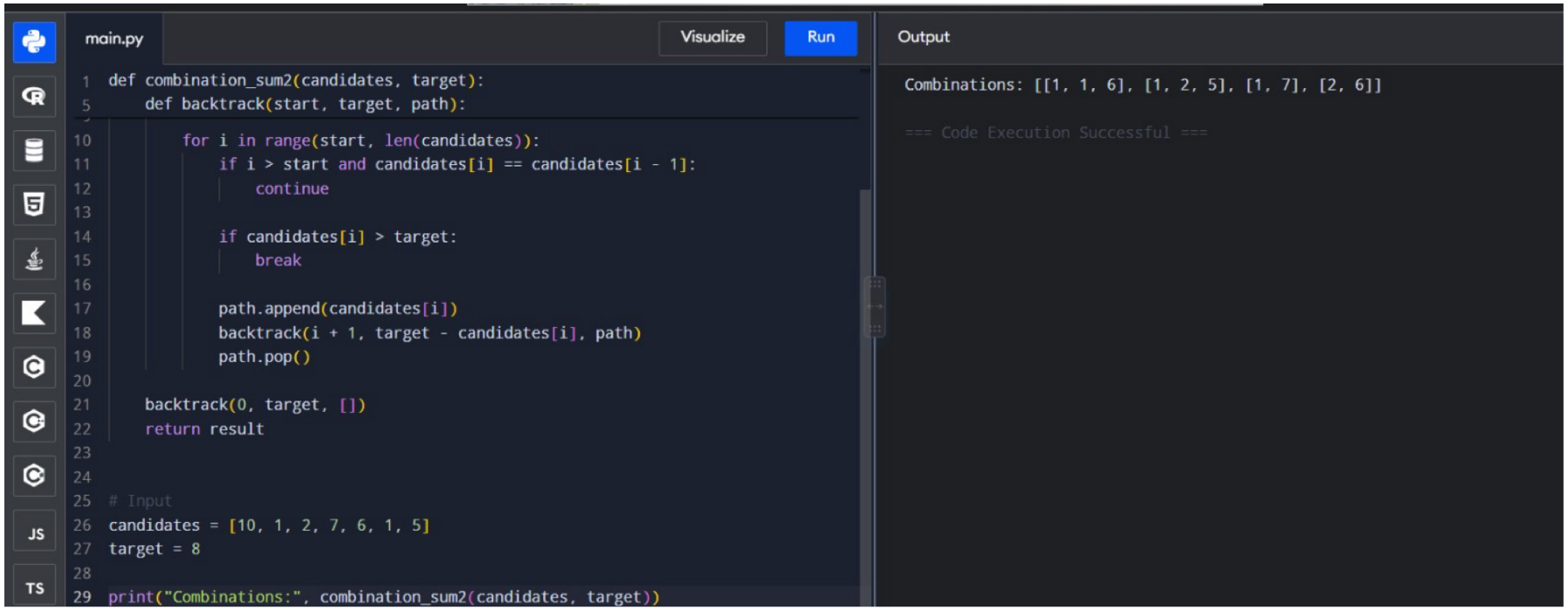

main.py
Visualize
Run
Output
def hamiltonian_cycle(n, edges):
def backtrack():
return False
if backtrack():
return True, path + [0]
return False, []
# Input
n = 5
edges = [
(0,1), (1,2), (2,3), (3,0),
(0,2), (2,4), (4,0)
]
exists, cycle = hamiltonian_cycle(n, edges)
print("Hamiltonian cycle exists:", exists)
if exists:
print("Cycle:", cycle)
Hamiltonian cycle exists: False
=== Code Execution Successful ===

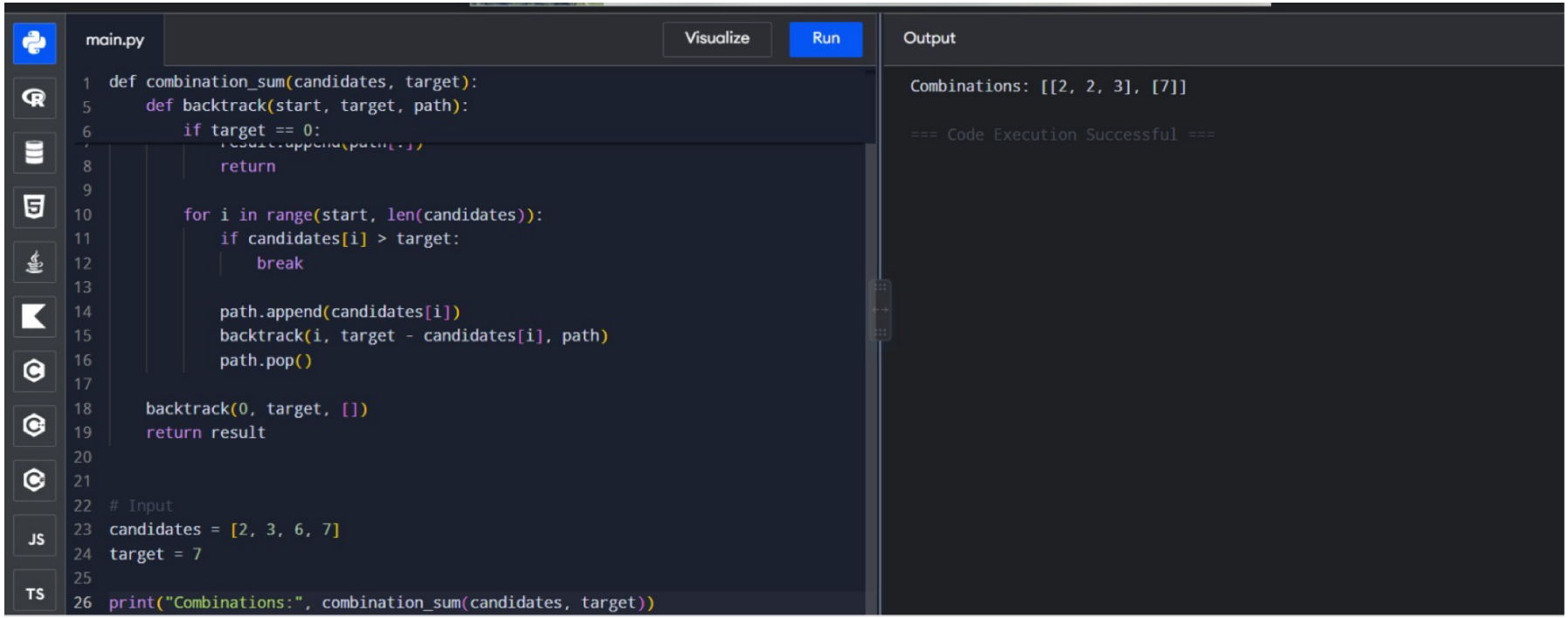

main.py
Visualize
Run
Output
def graph_coloring_k(n, edges, k):
def solve(v):
for c in range(1, k + 1):
if safe(v, c):
color[v] = 0
return False
if solve(0):
return color
return None
# Input
n = 4
k = 3
edges = [(0,1), (1,2), (2,3), (3,0), (0,2)]
answer = graph_coloring_k(n, edges, k)
print("Color assignment:", answer)
Color assignment: [1, 2, 3, 2]
=== Code Execution Successful ===

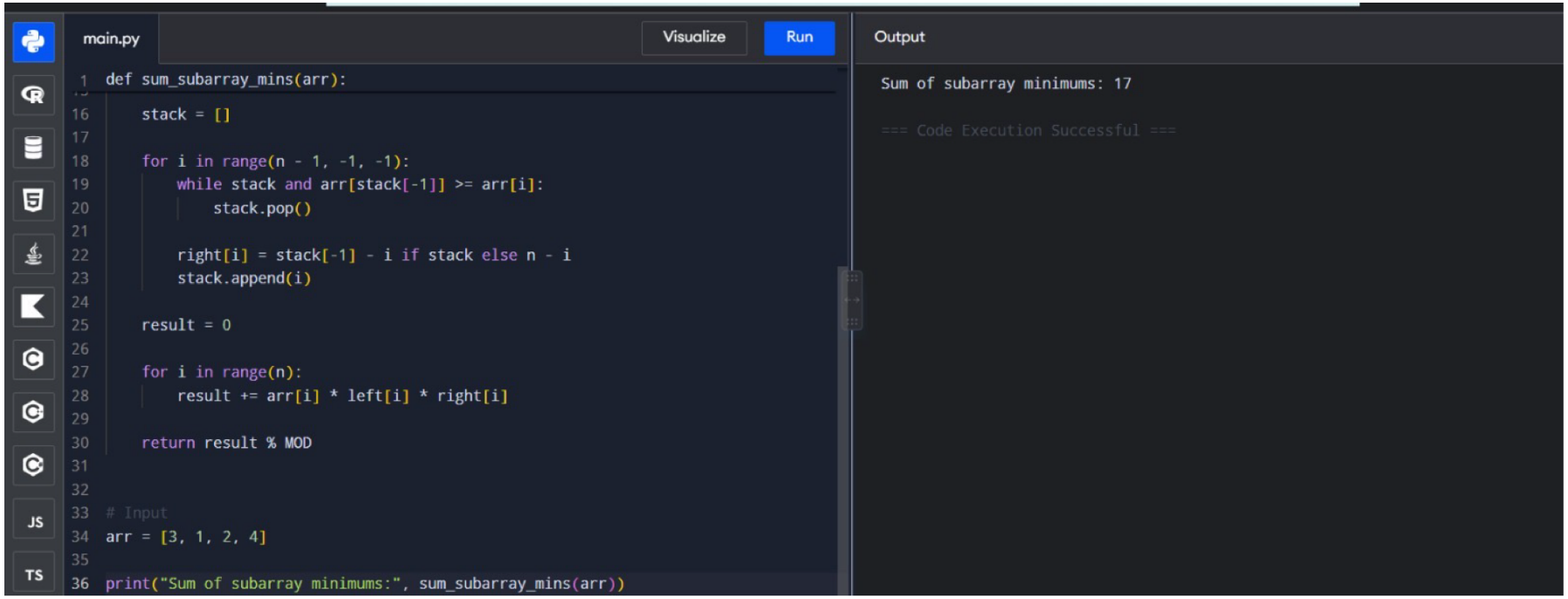

main.py
Visualize
Run
Output
def graph_coloring(n, edges):
def solve(v):
for c in range(1, n + 1):
if is_safe(v, c):
if solve(v + 1):
color[v] = -1
return False
solve(0)
return color
# Input
n = 4
edges = [(0,1), (1,2), (2,3), (3,0), (0,2)]
colors = graph_coloring(n, edges)
print("Vertex colors:", colors)
print("Number of colors:", max(colors))
Vertex colors: [1, 2, 3, 2]
Number of colors: 3
=== Code Execution Successful ===

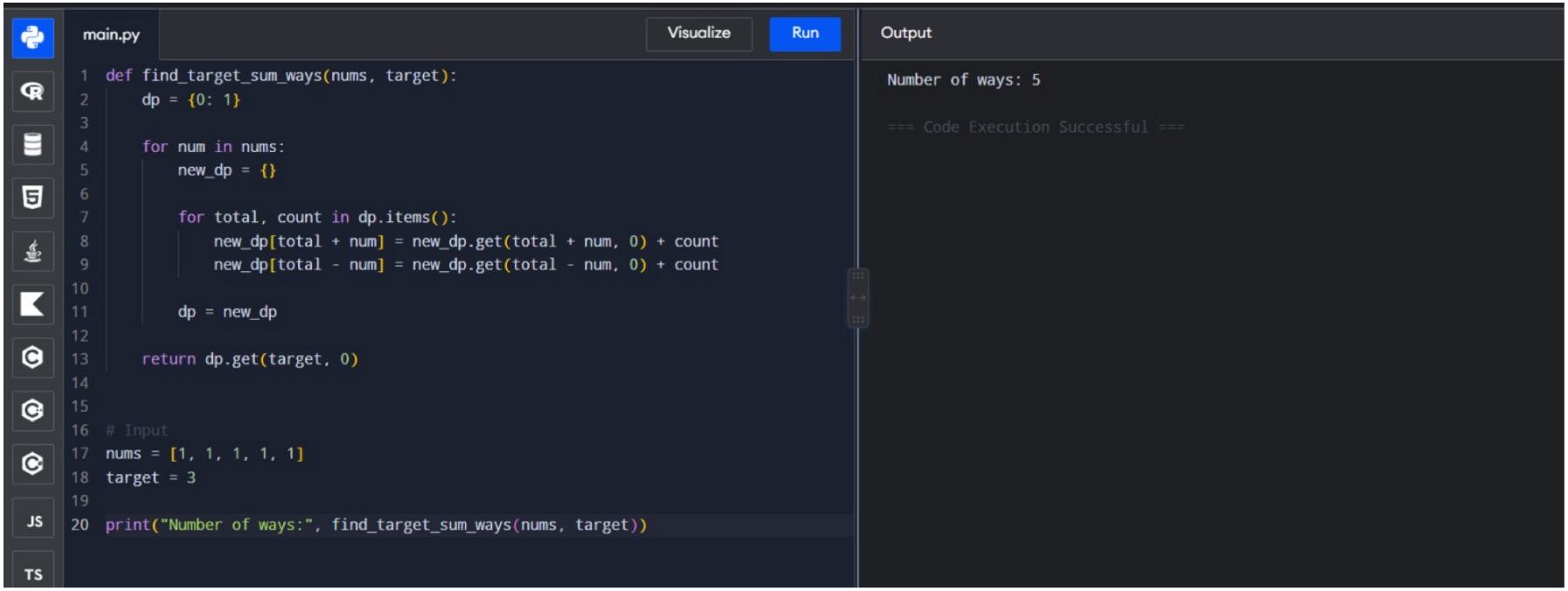

main.py
Visualize
Run
Output
def unique_permutations(nums):
def backtrack(path, used):
for i in range(len(nums)):
used[i] = True
path.append(nums[i])
backtrack(path, used)
path.pop()
used[i] = False
backtrack([], [False] * len(nums))
return result
# Input
nums = [1, 1, 2]
print("Unique permutations:")
for p in unique_permutations(nums):
print(p)
Unique permutations:
[1, 1, 2]
[1, 2, 1]
[2, 1, 1]
=== Code Execution Successful ===

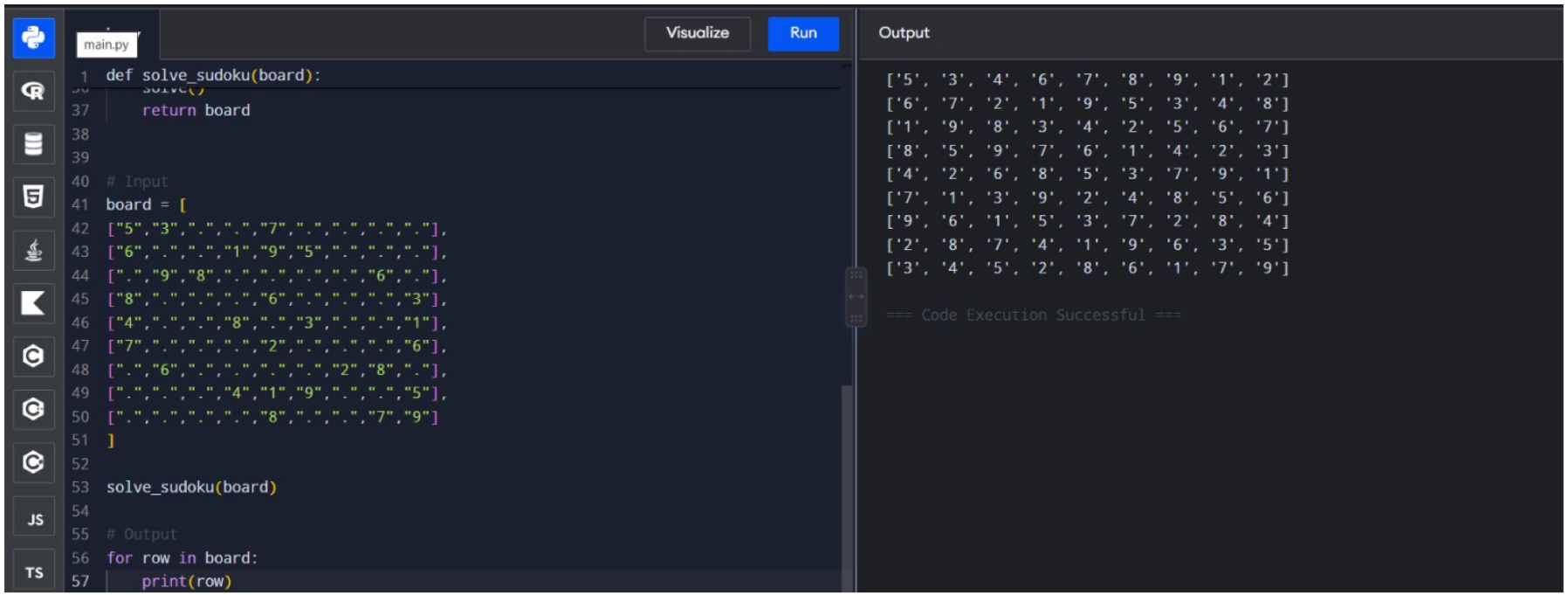

main.py
Visualize
Run
Output
def permutations(nums):
def backtrack(path, used):
for i in range(len(nums)):
if not used[i]:
used[i] = True
path.append(nums[i])
backtrack(path, used)
path.pop()
used[i] = False
backtrack([], [False] * len(nums))
return result
# Input
nums = [1, 2, 3]
print("Permutations:")
for p in permutations(nums):
print(p)
JS
TS
Permutations:
[1, 2, 3]
[1, 3, 2]
[2, 1, 3]
[2, 3, 1]
[3, 1, 2]
[3, 2, 1]
=== Code Execution Successful ===

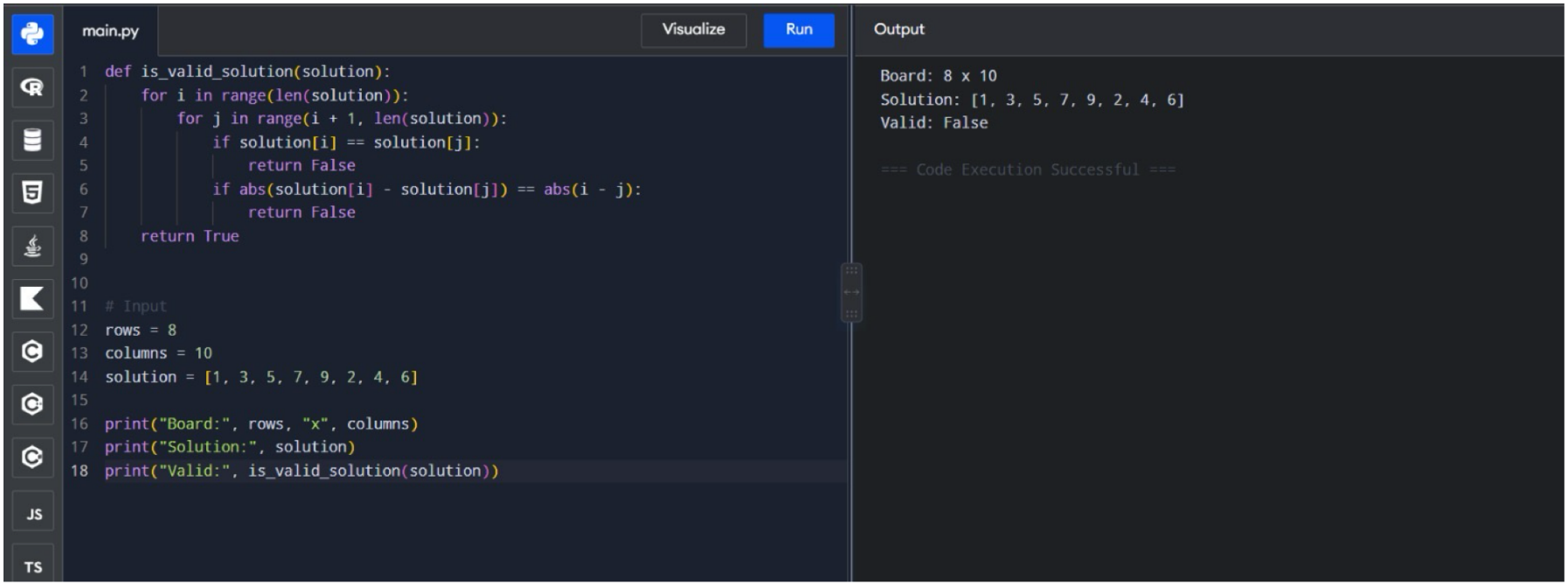

main.py
Visualize
Run
Output
def combination_sum2(candidates, target):
def backtrack(start, target, path):
for i in range(start, len(candidates)):
if i > start and candidates[i] == candidates[i - 1]:
continue
if candidates[i] > target:
break
path.append(candidates[i])
backtrack(i + 1, target - candidates[i], path)
path.pop()
backtrack(0, target, [])
return result
# Input
candidates = [10, 1, 2, 7, 6, 1, 5]
target = 8
print("Combinations:", combination_sum2(candidates, target))
Combinations: [[1, 1, 6], [1, 2, 5], [1, 7], [2, 6]]
=== Code Execution Successful ===

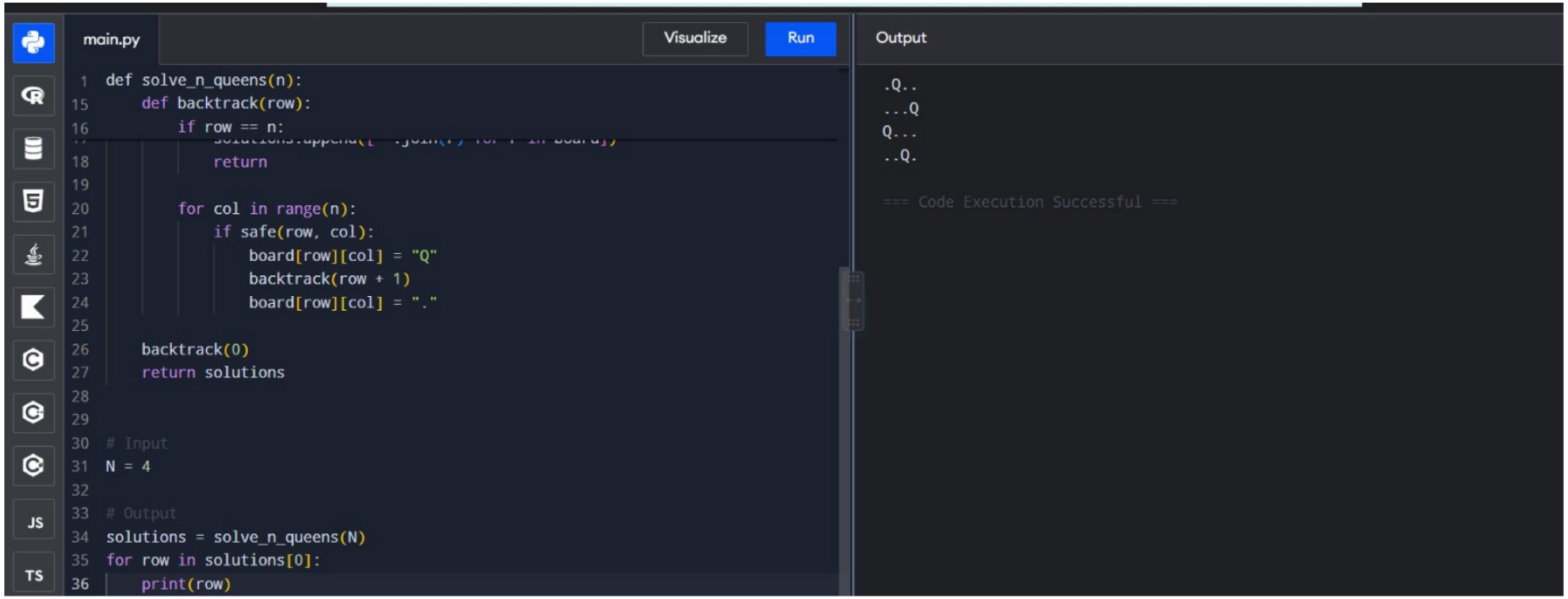

main.py
Visualize
Run
Output
def combination_sum(candidates, target):
def backtrack(start, target, path):
if target == 0:
return
for i in range(start, len(candidates)):
if candidates[i] > target:
break
path.append(candidates[i])
backtrack(i, target - candidates[i], path)
path.pop()
backtrack(0, target, [])
return result
# Input
candidates = [2, 3, 6, 7]
target = 7
print("Combinations:", combination_sum(candidates, target))
Combinations: [[2, 2, 3], [7]]
=== Code Execution Successful ===


main.py
Visualize
Run
Output
def sum_subarray_mins(arr):
stack = []
for i in range(n - 1, -1, -1):
while stack and arr[stack[-1]] >= arr[i]:
stack.pop()
right[i] = stack[-1] - i if stack else n - i
stack.append(i)
result = 0
for i in range(n):
result += arr[i] * left[i] * right[i]
return result % MOD
# Input
arr = [3, 1, 2, 4]
print("Sum of subarray minimums:", sum_subarray_mins(arr))
Sum of subarray minimums: 17
=== Code Execution Successful ===


main.py
Visualize
Run
Output
def find_target_sum_ways(nums, target):
dp = {0: 1}
for num in nums:
new_dp = {}
for total, count in dp.items():
new_dp[total + num] = new_dp.get(total + num, 0) + count
new_dp[total - num] = new_dp.get(total - num, 0) + count
dp = new_dp
return dp.get(target, 0)
# Input
nums = [1, 1, 1, 1, 1]
target = 3
print("Number of ways:", find_target_sum_ways(nums, target))
Number of ways: 5
=== Code Execution Successful ===


main.py
Visualize
Run
Output
def solve_sudoku(board):
return board
# Input
board = [
["5","3",".",".","7",".",".",".","."],
["6",".",".","1","9","5",".",".","."],
[".","9","8",".",".",".",".","6","."],
["8",".",".",".","6",".",".",".","3"],
["4",".",".","8",".","3",".",".","1"],
["7",".",".",".","2",".",".",".","6"],
[".","6",".",".",".",".","2","8","."],
[".",".",".","4","1","9",".",".","5"],
[".",".",".",".","8",".",".","7","9"]
]
solve_sudoku(board)
# Output
for row in board:
print(row)
['5', '3', '4', '6', '7', '8', '9', '1', '2']
['6', '7', '2', '1', '9', '5', '3', '4', '8']
['1', '9', '8', '3', '4', '2', '5', '6', '7']
['8', '5', '9', '7', '6', '1', '4', '2', '3']
['4', '2', '6', '8', '5', '3', '7', '9', '1']
['7', '1', '3', '9', '2', '4', '8', '5', '6']
['9', '6', '1', '5', '3', '7', '2', '8', '4']
['2', '8', '7', '4', '1', '9', '6', '3', '5']
['3', '4', '5', '2', '8', '6', '1', '7', '9']
=== Code Execution Successful ===


main.py
Visualize
Run
Output
def is_valid_solution(solution):
for i in range(len(solution)):
for j in range(i + 1, len(solution)):
if solution[i] == solution[j]:
return False
if abs(solution[i] - solution[j]) == abs(i - j):
return False
return True
# Input
rows = 8
columns = 10
solution = [1, 3, 5, 7, 9, 2, 4, 6]
print("Board:", rows, "x", columns)
print("Solution:", solution)
print("Valid:", is_valid_solution(solution))
Board: 8 x 10
Solution: [1, 3, 5, 7, 9, 2, 4, 6]
Valid: False
=== Code Execution Successful ===
JS
TS


main.py
Visualize
Run
Output
def solve_n_queens(n):
def backtrack(row):
if row == n:
return
for col in range(n):
if safe(row, col):
board[row][col] = "Q"
backtrack(row + 1)
board[row][col] = "."
backtrack(0)
return solutions
# Input
N = 4
# Output
solutions = solve_n_queens(N)
for row in solutions[0]:
print(row)
.Q..
...Q
Q...
..Q.
=== Code Execution Successful ===